

[项目实战]在线教育-采集阶段

练习题

习题一 作为开发智囊

你可以尝试让 GPT 扮演大数据工程师，在开发过程中为你提供开发锦囊。

习题二 编写启停脚本

请尝试使用 GPT 编写 Kafka 集群的启停脚本。

习题三 编写 DataX 批量调度脚本

请尝试使用 GPT 编写课程中用于 DataX 批量调度的 Bash 脚本。

参考答案

习题一 作为开发智囊

第 1 轮提示

提示

你现在是一个经验丰富的数据仓库工程师，我们现在要搭建一个业务数据库的采集平台，当我说出一个表的表名，和描述，你要判断一下它适合做增量同步还是全量同步，或者增量全量都应该做。并且你要给出详细的原因说明

答

当然可以。请告诉我表名和相关描述，我将为您分析并给出建议。

反馈

可以看到，GPT 已经进入状态。现在，我们可以尝试问一些数据同步相关的问题。

第 2 轮提示

提示

省份表

答

对于省份表，我建议使用全量同步。

原因如下：

1. 数据量小：中国的省份数量相对较少，数据量不大，因此全量同步不会产生太大的性能开销。
2. 数据变动较少：省份数据相对稳定，很少发生变动。即使有变动，也是非常低频的事件。因此，使用全量同步可以保证每次同步都能获取最新的省份数据。
3. 简化同步逻辑：全量同步相对于增量同步来说，同步逻辑更加简单，易于维护。因为省份数据变动较少，使用全量同步可以简化同步过程，减少潜在的错误和维护成本。

总之，基于以上原因，对于省份表这种数据量小且变动较少的表，全量同步是更合适的选择。

反馈

这是一个简单的示例。

第 3 轮提示

提示

课程信息表，里面每行记录都是一个课程的信息，全表数据量大概在 5000 左右，里面的数据很少更新，我需要每天计算一次平台上所有课程的结课总人数。

答

对于这个课程信息表，我建议使用全量同步。

原因如下：

1. 数据量适中：全表数据量为 5000 条，相对较少，使用全量同步在每次同步时处理的数据量依然可以接受，不会产生太大的性能开销。
2. 数据更新较少：您提到课程信息表中的数据很少更新，这意味着大部分时间，增量同步和全量同步的结果并无太大差异。在这种情况下，全量同步可以简化同步逻辑，降低维护成本。
3. 确保数据准确性：由于您每天需要计算平台上所有课程的结课总人数，使用全量同步可以确保每次同步都能获取完整的课程信息，避免因增量同步导致的数据不一致问题。

总之，基于以上原因，对于这个数据量适中且数据更新较少的课程信息表，全量同步是更合适的选择。

反馈

可以接受

第 4 轮提示

提示

章节评价表，里面有平台上所有章节的所有课程的评价，全表每天大概新增 2000 条数据，我需要统计最近 1/7/30 日的各课程的用户平均评分，评价用户数等

答

对于章节评价表，我建议使用增量同步。

原因如下：

1. 数据量较大并持续增长：根据您的描述，每天会新增 2000 条数据。随着时间的推移，数据量会持续增长，使用全量同步会导致同步时间和计算资源的开销逐渐增加。
2. 高频更新：章节评价表每天都有新增数据，这意味着数据的变动较为频繁。使用增量同步可以确保每次同步只处理新增的评价数据，提高同步效率。
3. 针对性统计：您需要统计最近 1/7/30 日的各课程的用户平均评分和评价用户数，增量同步可以满足这个需求。每天同步新增的评价数据，然后根据需要对这部分数据进行统计分析即可，避免对全部数据进行重复计算。

综上所述，针对章节评价表这种数据量较大、持续增长且高频更新的场景，增量同步是更合适的选择。这样既可以提高同步效率，又可以降低计算资源的消耗。

反馈

可以看到，GPT 能够给出较好的同步建议，但是需要注意，GPT 能给出较好的建议，是建立在信息充足的基础之上的。你可以尝试一下，如果你给出较少的提示，GPT 大多数情况给出的建议都是做增量同步。

习题二 编写启停脚本

第 1 轮提示

提示

我有三台机器，hadoop102, hadoop103, hadoop104。请帮我编写一个 bash 脚本管理 3 个节点上 kafka 的启停。3 台机器上的 kafka 安装路径都是一样的

答

以下是一个简单的 bash 脚本示例，可以用来在 hadoop102、hadoop103 和 hadoop104 上管理 Kafka 服务的启动和停止。我假设 Kafka 的启动脚本是在 Kafka 的安装目录中，路径是 `/opt/kafka/bin/kafka-server-start.sh`，而停止脚本是 `/opt/kafka/bin/kafka-server-stop.sh`。

注意：使用此脚本前，请确保已在三台机器上设置了 ssh 无密码登录，这样我们才能远程执行命令。

```
#!/bin/bash

# 定义节点列表
nodes=("hadoop102" "hadoop103" "hadoop104")

# Kafka 安装路径
kafka_path="/opt/kafka/bin"

# 启动 kafka
start_kafka() {
    for node in "${nodes[@]"; do
        echo "正在 $node 启动 Kafka..."
        ssh $node "nohup $kafka_path/kafka-server-start.sh > /dev/null 2>&1 &"
    done
}
```

```
# 停止 kafka
stop_kafka() {
    for node in "${nodes[@]"}; do
        echo "正在 $node 停止 Kafka..."
        ssh $node "$kafka_path/kafka-server-stop.sh"
    done
}

# 根据参数执行相应操作
case $1 in
    "start")
        start_kafka
        ;;
    "stop")
        stop_kafka
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        exit 1
        ;;
esac
```

这个脚本通过 SSH 连接到每台机器并执行相应的启动或停止命令。将此脚本保存为文件（例如 `manage_kafka.sh`），然后使其可执行（`chmod +x manage_kafka.sh`），之后你可以通过调用 `./manage_kafka.sh start` 来启动 kafka，或者通过调用 `./manage_kafka.sh stop` 来停止 kafka。

此脚本使用了 Kafka 的默认启动和停止脚本。如果你的环境或安装路径不同，你可能需要相应地修改这些路径。同样，如果你有更多的节点，你也可以通过添加到 `nodes` 数组来支持。

警告：在实际环境中，您应该小心使用此类脚本，确保在所有环境中都已经正确测试过。

反馈

很棒，GPT 编写的脚本有一个特点，就是便于修改。它会把 `path`, `nodes` 这些变量单独声明在脚本的最开始处。假设，我们现在又想加一个功能，比如查看 `kafka` 当前的工作状态，那么我们还可以让 GPT 继续迭代这个脚本。

第 2 轮提示

提示

我希望再添加一个子命令 `status`，可以用来检查 `Kafka` 的状态是否正常

答

你可以通过检查 `Kafka` 进程是否在运行来简单地判断 `Kafka` 的状态。我们可以使用 `jps` 命令，这是 `Java` 的一个工具，可以列出 `JVM` 的进程。

下面的脚本增加了一个名为“`status`”的子命令，它会在每个节点上运行 `jps` 命令并查找 `Kafka` 相关的进程：

```
#!/bin/bash

# 定义节点列表
nodes=("hadoop102" "hadoop103" "hadoop104")

# Kafka 安装路径
kafka_path="/opt/kafka/bin"

# 启动 kafka
start_kafka() {
    for node in "${nodes[@]"; do
        echo "正在 $node 启动 Kafka..."
```

```
        ssh $node "nohup $kafka_path/kafka-server-start.sh > /dev/null 2>&1 &"
    done
}

# 停止 kafka
stop_kafka() {
    for node in "${nodes[@]"; do
        echo "正在 $node 停止 Kafka..."
        ssh $node "$kafka_path/kafka-server-stop.sh"
    done
}

# 检查 kafka 状态
check_kafka_status() {
    for node in "${nodes[@]"; do
        echo "检查 $node 上的 Kafka 状态..."
        ssh $node "jps | grep -i kafka"
    done
}

# 根据参数执行相应操作
case $1 in
    "start")
        start_kafka
        ;;
    "stop")
        stop_kafka
        ;;
    "status")
        check_kafka_status
        ;;
    *)
        echo "Usage: $0 {start|stop|status}"
        exit 1
        ;;
esac
```


通过执行 `./manage_kafka.sh status`，你可以检查 Kafka 的状态。如果 Kafka 正在运行，你会看到一个包含“kafka”的进程行。如果没有看到这样的进程，那么 Kafka 可能没有运行。

注意，这只是一种基本的健康检查方式，仅仅检查了进程是否存在。在实际情况中，Kafka 的健康状态可能需要更复杂的检查，比如检查日志文件，监控系统的资源使用情况，或者通过 Kafka 自身的管理 API 进行检查等。

反馈

只需稍微修改 kafka 安装路径即可使用。

习题三 编写 DataX 批量调度脚本

第 1 轮提示

提示

我在 linux 的一个目录下放了很多的 datax 同步配置文件，它们都是按照表名命名的 .json，而且功能都是从 mysql 读取数据同步到 hadoop 的指定目录下

现在，我想写一个配置脚本，它支持以下 flag

第一个 flag `--tables`

tables 后可以跟多个值，它们都是表名。然后这个 bash 脚本要先检查这些 `<table>.json` 是不是在目录里存在，如果有一个 `<table>.json` 不存在就报错。如果存在，就做数据同步操作。

第二个 flag `--all` `--all` 和 `--tables` 不能同时指定

all 不需要传递具体值，它会去目录下找到所有的 .json 结尾的文件依次做数据同步操作。

第三个 flag `--date`

date 可选，如果没有传递的话，默认是执行时的前一天的日期。

第四个 flag --help

可以打印所有此 bash 脚本的用法

数据同步操作的具体流程是

首先确定数据的目标 hdfs 路径，这个路径的格式是/origin_data/warehouse/db/\${表名}/\${date}

1. 使用 hadoop 尝试删除这个路径，然后再重新创建这个路径
2. 利用 datax 执行数据同步操作，这个时候需要传递一个参数，就是之前的目标路径

答

根据你的需求，以下是一个简单的 bash 脚本来完成这个任务：

```
#!/bin/bash

TABLES=()
ALL=false
DATE=$(date -d "yesterday" +%Y-%m-%d)

usage() {
    echo "Usage: $0 [--tables table1 table2 ...] [--all] [--date YYYY-MM-DD]"
    echo "--tables: List of tables to be synchronized."
    echo "--all: Synchronize all tables."
    echo "--date: Optional. Specify the date. Default is the day before the script is run."
    echo "--help: Display this help."
    exit 1
}

while [[ "$#" -gt 0 ]]; do
    case $1 in
        --tables)
```

```

        shift
    while [[ "$#" -gt 0 && "$1" != "--"* ]]; do
        TABLES+=("$1")
        shift
    done
    ;;
--all)
    ALL=true
    shift
    ;;
--date)
    shift
    DATE="$1"
    shift
    ;;
--help)
    usage
    ;;
*)
    echo "Unknown parameter: $1"
    usage
    ;;
esac

done

if $ALL; then
    TABLES=$(ls *.json | cut -f 1 -d '.')
```



```

fi

for table in "${TABLES[@]"; do
    if [ ! -f "${table}.json" ]; then
        echo "Error: Configuration file for table ${table} not found."
        exit 1
    fi

    hdfs_path="/origin_data/warehouse/db/${table}/${DATE}"

```

```
# 删除旧目录并创建新目录
hadoop fs -rm -r $hdfs_path
hadoop fs -mkdir -p $hdfs_path

# 执行数据同步操作
datax -json ${table}.json -parameter "targetPath=$hdfs_path"
done
```

这个脚本的工作方式如下：

- 从命令行参数中获取所需的表，全部同步标志，和可选的日期。
- 如果指定了--all 标志，那么它将获取目录中所有.json 文件的名称作为表名。
- 对于每一个表，如果对应的.json 配置文件不存在，那么将会打印错误信息并退出。
- 对于每一个表，它将删除并重新创建 HDFS 路径，然后调用 datax 进行数据同步。

请注意，你需要根据你的环境和实际的 datax 命令行参数进行必要的修改。例如，`datax -json ${table}.json -parameter "targetPath=$hdfs_path"` 这行可能需要调整，因为我不确定你的 DataX 具体如何接收参数。

反馈

已经非常准确了，只是最后 datax 的调用并不准确，gpt 也说出了它不知道 dataX 传参的具体方式。这个时候为了进一步迭代脚本，我们可以去服务器上调一下 datax.py 脚本，打印它的帮助信息：

```

[atguigu@hadoop102 bin]$ ./datax.py
DataX (DATAX-OPENSOURCE-3.0), From Alibaba !
Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

Usage: datax.py [options] job-url-or-path

Options:
  -h, --help            show this help message and exit

Product Env Options:
  Normal user use these options to set jvm parameters, job runtime mode
  etc. Make sure these options can be used in Product Env.

  -j <jvm parameters>, --jvm=<jvm parameters>
                        Set jvm parameters if necessary.
  --jobid=<job unique id>
                        Set job unique id when running by Distribute/Local
                        Mode.
  -m <job runtime mode>, --mode=<job runtime mode>
                        Set job runtime mode such as: standalone, local,
                        distribute. Default mode is standalone.
  -p <parameter used in job config>, --params=<parameter used in job config>
                        Set job parameter, eg: the source tableName you want
                        to set it by command, then you can use like this:
                        -p"-DtableName=your-table-name", if you have mutiple
                        parameters: -p"-DtableName=your-table-name
                        -DcolumnName=your-column-name".Note: you should config
                        in you job tableName with ${tableName}.
  -r <parameter used in view job config[reader] template>, --reader=<parameter used in view job config[reader] template>
                        View job config[reader] template, eg:
                        mysqlreader,streamreader
  -w <parameter used in view job config[writer] template>, --writer=<parameter used in view job config[writer] template>
                        View job config[writer] template, eg:
                        mysqlwriter,streamwriter

Develop/Debug Options:
  Developer use these options to trace more details of DataX.

  -d, --debug           Set to remote debug mode.
  --loglevel=<log level>
                        Set log level such as: debug, info, all etc.

```

第 2 轮提示

提示

在 datax 的安装目录下有一个 bin 子目录，里面有一个 datax.py 脚本，它是实际的 datax 命令

下面是 datax.py 打印的帮助信息

DataX (DATAX-OPENSOURCE-3.0), From Alibaba !

Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

Usage: datax.py [options] job-url-or-path

Options:

-h, --help show this help message and exit

Product Env Options:

Normal user use these options to set jvm parameters, job runtime mode

etc. Make sure these options can be used in Product Env.

`-j <jvm parameters>, --jvm=<jvm parameters>`

Set jvm parameters if necessary.

`--jobid=<job unique id>`

Set job unique id when running by Distribute/Local

Mode.

`-m <job runtime mode>, --mode=<job runtime mode>`

Set job runtime mode such as: standalone, local,

distribute. Default mode is standalone.

`-p <parameter used in job config>, --params=<parameter used in job config>`

Set job parameter, eg: the source tableName you want

to set it by command, then you can use like this:

`-p"-DtableName=your-table-name"`, if you have mutiple

parameters: `-p"-DtableName=your-table-name`

`-DcolumnName=your-column-name"`.Note: you should config

in you job tableName with `${tableName}`.

`-r <parameter used in view job config[reader] template>, --reader=<parameter used in view job config[reader] template>`

View job config[reader] template, eg:

mysqlreader, streamreader

`-w <parameter used in view job config[writer] template>, --writer=<parameter used in view job config[writer] template>`

View job config[writer] template, eg:

mysqlwriter, streamwriter

Develop/Debug Options:

Developer use these options to trace more details of DataX.

`-d, --debug` Set to remote debug mode.

`--loglevel=<log level>`

Set log level such as: debug, info, all etc.

请修改之前的 bash 脚本

答

感谢你提供的详细信息，下面是根据你的需求和 DataX 的命令行参数修改后的 bash 脚本：

```
#!/bin/bash

TABLES=()
ALL=false
DATE=$(date -d "yesterday" +%Y-%m-%d)
DATA_HOME="/path/to/datax" # 这里需要填写你的 DataX 的安装目录

usage() {
    echo "Usage: $0 [--tables table1 table2 ...] [--all] [--date YYYY-MM-DD]"
    echo "--tables: List of tables to be synchronized."
    echo "--all: Synchronize all tables."
    echo "--date: Optional. Specify the date. Default is the day before the script is run."
    echo "--help: Display this help."
}
```

```

    exit 1
}

while [[ "$#" -gt 0 ]]; do
    case $1 in
        --tables)
            shift
            while [[ "$#" -gt 0 && "$1" != "--"* ]]; do
                TABLES+=("$1")
                shift
            done
            ;;
        --all)
            ALL=true
            shift
            ;;
        --date)
            shift
            DATE="$1"
            shift
            ;;
        --help)
            usage
            ;;
        *)
            echo "Unknown parameter: $1"
            usage
            ;;
    esac
done

if $ALL; then
    TABLES=$(ls *.json | cut -f 1 -d '.')
fi

for table in "${TABLES[@]"; do

```



```

if [ ! -f "${table}.json" ]; then
    echo "Error: Configuration file for table ${table} not found."
    exit 1
fi

hdfs_path="/origin_data/warehouse/db/${table}/${DATE}"

# 删除旧目录并创建新目录
hadoop fs -rm -r $hdfs_path
hadoop fs -mkdir -p $hdfs_path

# 执行数据同步操作
${DATAX_HOME}/bin/datax.py -j "-DtargetPath=$hdfs_path" ${table}.json
done

```

在这个版本的脚本中，我增加了一个变量 **DATAX_HOME** 来代表 DataX 的安装目录，并且修改了 DataX 命令行的调用方式，现在会使用 **-j** 选项来设置 **targetPath** JVM 参数。

注意：你需要替换 **DATAX_HOME** 为你的实际 DataX 安装路径，并且在你的 .json 配置文件中，你需要使用 **"\${targetPath}"** 这样的方式来引用 **targetPath** 参数。

这个脚本还需要根据你的环境和需求进行修改和调试。使用前，请务必仔细检查和测试，确保它适合你的环境和需求。

反馈

现在已经基本满足要求了，只是我们一次抛出的信息过长，GPT 误将参数的 flag 弄成了 **-j**，我们修改为 **-p** 即可。



```

hadoop fs -rm -r $hdfs_path
hadoop fs -mkdir -p $hdfs_path

# 执行数据同步操作
${DATAX_HOME}/bin/datax.py -p "-DtargetPath=$hdfs_path" ${table}.json

```